



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 10
по курсу «Языки и методы программирования»
«Реализация итераторов на языке C++»

Студент группы ИУ9-21Б Белянин Г. И.

Преподаватель Посевин Д. П.

Москва 2026

1 Цель

Данная работа предназначена для приобретения навыков разработки контейнерных классов с итераторам.

2 Персональный вариант

Представление числа в фибоначчией системе счисления с константным однонаправленным итератором по разрядам представления.

3 Практическая реализация

Main

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <stdexcept>
#include <iterator>

class FibonacciRepresentation {
private:
    std::vector<int> digits;

    static const std::vector<int>& getFibSequence() {
        static std::vector<int> fibs;
        if (fibs.empty()) {
            long long a = 1, b = 2;
            while (a <= 2147483647LL) {
                fibs.push_back(static_cast<int>(a));
                long long next = a + b;
                a = b;
                b = next;
            }
        }
        return fibs;
    }

public:
    explicit FibonacciRepresentation(int value) {
        if (value < 0) throw std::invalid_argument(" ");

        if (value == 0) {
            digits = {0};
            return;
        }

        const auto& fibs = getFibSequence();
        int n = value;
        bool started = false;
        for (int i = static_cast<int>(fibs.size()) - 1; i >= 0; --i) {
            if (n >= fibs[i]) {
                digits.push_back(1);
                n -= fibs[i];
                started = true;
            } else if (started) {
                digits.push_back(0);
            }
        }
    }
};
```

```

    }
}

class const_iterator {
private:
    std::vector<int>::const_iterator internal_it;

public:
    using iterator_category = std::forward_iterator_tag;
    using value_type        = int;
    using difference_type    = std::ptrdiff_t;
    using pointer            = const int*;
    using reference          = const int&;

    explicit const_iterator(std::vector<int>::const_iterator p) :
        internal_it(p) {}

    reference operator*() const { return *internal_it; }
    pointer operator->() const { return &(*internal_it); }

    const_iterator& operator++() {
        ++internal_it;
        return *this;
    }

    const_iterator operator++(int) {
        const_iterator tmp = *this;
        ++(*this);
        return tmp;
    }

    friend bool operator==(const const_iterator& a, const const_iterator&
        b) {
        return a.internal_it == b.internal_it;
    }
    friend bool operator!=(const const_iterator& a, const const_iterator&
        b) {
        return !(a == b);
    }
};

const_iterator begin() const { return const_iterator(digits.begin()); }
const_iterator end()   const { return const_iterator(digits.end()); }

```

```

        friend std::ostream& operator<<(std::ostream& os, const
            FibonacciRepresentation& rep) {
            for (int d : rep.digits) os << d;
            return os;
        }
};

int main() {
    try {
        std::vector<int> test_numbers = {1, 2, 3, 5, 9, 19, 21, 34, 70, 100};

        for (int n : test_numbers) {
            FibonacciRepresentation fib_num(n);

            std::cout << n << ": ";
            for (const int bit : fib_num) {

                std::cout << bit;
            }
            std::cout << std::endl;
        }
    } catch (const std::exception& e) {
        std::cerr << e.what() << std::endl;
    }
    return 0;
}

```

4 Результаты

```

1: 1
2: 10
3: 100
5: 1000
9: 10001
19: 101001
21: 1000000
34: 10000000
70: 100100010
100: 1000010100

```

Program ended with exit code: 0

5 Выводы

В ходе выполнения лабораторной работы был разработан класс `FibonacciRepresentation`, реализующий представление целого числа в фибоначчической системе счисления. Освоены принципы создания пользовательских контейнеров и итераторов в языке C++.